

glmCH2_extra

Create dataframe where each row represents number of queens and workers seen during each survey event

Load dataframes

```
#load packages
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(lubridate) #package used to generate julian date from calendar date

#read in data
bee <- read.csv("06_cleandata/bee_clean.csv")
floral <- read.csv("06_cleandata/floral_clean.csv")
rodent <- read.csv("06_cleandata/rodent_clean.csv")
veg <- read.csv("06_cleandata/veg_clean.csv")
site_info <- read.csv("06_cleandata/site_info_clean.csv")
surveys <- read.csv("06_cleandata/surveys.csv")
```

Fix dataframe structure

```
#bee
str(bee)

## 'data.frame':    520 obs. of  17 variables:
## $ date          : chr  "2024-04-13" "2024-04-13" "2024-04-13" "2024-04-13" ...
## $ observers      : chr  "RG" "RG" "RG" "RG" ...
## $ site_id        : chr  "chong_96" "chong_96" "chong_96" "chong_96" ...
## $ round          : chr  "2024_r1" "2024_r1" "2024_r1" "2024_r1" ...
```

```
## $ plot_id      : chr "p1" "p2" "d" "d" ...
## $ transect_id  : chr "full-plot" "full-plot" "full-plot" "full-plot" ...
## $ transect_section: chr "net-floral" "net-floral" "net-floral" "net-floral" ...
## $ sample_code  : chr "chong_96_NS_p1_full-plot_net-floral" "chong_96_NS_p2_full-plot_net-floral"
## $ behaviour    : chr "flying" "flying" "flying" "nest_searching" ...
## $ caste        : chr "q" "q" "q" "q" ...
## $ flower_code   : chr "" "" "" "" ...
## $ flower_species : chr "" "" "" "" ...
## $ bee_code      : chr "" "" "boim" "bovo" ...
## $ bee_species   : chr "" "" "bombus_impatiens" "bombus_vosnesenskii" ...
## $ caught       : chr "n" "n" "y" "n" ...
## $ photo_series  : chr "" "" "" "" ...
## $ notes        : chr "could not ID" "could not ID" "" "" ...
```

```
bee$date <- ymd(bee$date)
```

```
#floral
str(floral)
```

```
## 'data.frame': 786 obs. of 12 variables:
## $ date          : chr "2024-04-13" "2024-04-16" "2024-04-16" "2024-04-18" ...
## $ observers     : chr "RG" "RG" "CN" "RG" ...
## $ site_id       : chr "chong_96" "bates_57b" "bates_57b" "bks_96" ...
## $ round         : chr "2024_r1" "2024_r1" "2024_r1" "2024_r1" ...
## $ plot_id       : chr "p2" "p1" "p2" "p1" ...
## $ transect_id   : chr "full-plot" "full-plot" "full-plot" "full-plot" ...
## $ transect_section: chr "net-floral" "net-floral" "net-floral" "net-floral" ...
## $ sample_code   : chr "chong_96_NS_p2_full-plot_net-floral" "bates_57b_NS_p1_full-plot_net-floral"
## $ flower_code   : chr "taof" "taof" "taof" "vaco" ...
## $ flower_species : chr "taraxacum_officinale" "taraxacum_officinale" "taraxacum_officinale" "vaccinium"
## $ abundance     : int 2 4 4 4 1 4 3 4 4 2 ...
## $ notes        : chr "" "" "" "" ...
```

```
floral$date <- ymd(floral$date)
```

```
#rodent
str(rodent)
```

```
## 'data.frame': 1037 obs. of 25 variables:
## $ date_day      : chr "2024-04-17" "2024-04-17" "2024-04-17" "2024-04-17" ...
## $ observers_day  : chr "RG, AD" "RG, AD" "RG, AD" "RG, AD" ...
## $ date_night    : chr "" "" "" "" ...
## $ observers_night : chr "" "" "" "" ...
## $ site_id       : chr "gwb_neaves" "gwb_neaves" "gwb_neaves" "gwb_neaves" ...
## $ round         : chr "2024_r1" "2024_r1" "2024_r1" "2024_r1" ...
## $ plot_id       : chr "d" "d" "d" "d" ...
## $ transect_id   : chr "t2" "t2" "t2" "t2" ...
## $ transect_section : chr "rod-s1" "rod-s1" "rod-s1" "rod-s1" ...
## $ burrow_code    : chr "gwb_neaves_d_t2_rod-s1" "gwb_neaves_d_t2_rod-s1" "gwb_neaves_d_t2_rod-s1"
## $ location_m     : num 0 0.2 0.4 0.5 5 7.5 6 5 20 20.5 ...
## $ location_side  : chr "" "" "" "" ...
## $ row_alley      : logi NA NA NA NA NA NA ...
## $ diameter_cm    : num 2.5 5 2.5 5 2.5 2.5 2.5 2.5 5 5 ...
```

```
## $ species_guess      : chr  "" "" "" "" ...
## $ substrate          : chr  "" "" "" "" ...
## $ notes_hole         : chr  "rocky" "rocky" "rocky" "rocky" ...
## $ powder_active_field : chr  "" "" "" "" ...
## $ powder_species_guess_field: chr  "" "" "" "" ...
## $ notes_powder       : chr  "" "" "" "" ...
## $ photo_number       : chr  "" "" "" "" ...
## $ photo_location     : chr  "" "" "" "" ...
## $ powder_active_FINAL : chr  "" "" "" "" ...
## $ powder_species_guess_FINAL: chr  "" "" "" "" ...
## $ notes_extra_FINAL  : chr  "" "" "" "" ...
```

```
rodent$date_day <- ymd(rodent$date_day)
```

```
#veg
str(veg)
```

```
## 'data.frame':    2802 obs. of  128 variables:
## $ date           : chr  "2024-04-13" "2024-04-13" "2024-04-13" "2024-04-13" ...
## $ observers      : chr  "RG" "RG" "RG" "RG" ...
## $ site_id        : chr  "chong_96" "chong_96" "chong_96" "chong_96" ...
## $ round          : chr  "2024_r1" "2024_r1" "2024_r1" "2024_r1" ...
## $ plot_id        : chr  "p2" "p2" "p2" "p2" ...
## $ transect_id    : chr  "t1" "t2" "t1" "t2" ...
## $ quadrat        : chr  "veg-q1m" "veg-q1e" "veg-q3m" "veg-q1m" ...
## $ sample_code    : chr  "chong_96_NS_p2_t1_veg-q1m" "chong_96_NS_p2_t2_veg-q1e" "chong_96_NS_p2_t1_veg-q1m" ...
## $ bare           : num  23 25 35 35 35 35 35 45 45 50 ...
## $ dist           : int  0 0 0 0 0 0 0 0 0 0 ...
## $ grass          : num  70 2 60 45 60 10 0 10 0.5 20 ...
## $ forb           : num  5 4 0 0 0 0 0 0 0 0 ...
## $ grass_height   : num  10 7 16 14 14 8 NA 7 3 10 ...
## $ forb_height    : num  6 4 NA NA NA NA NA NA NA NA ...
## $ mulch          : int  0 50 0 0 0 45 45 35 50 0 ...
## $ dry_root       : num  2 20 5 20 5 10 20 10 5 30 ...
## $ flood          : num  0 0 0 0 0 0 0 0 0 0 ...
## $ dry.unroot     : int  NA NA NA NA NA NA NA NA NA NA ...
## $ bryophyte      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ live_mow       : int  NA NA NA NA NA NA NA NA NA NA ...
## $ rock           : int  0 0 0 0 0 0 0 0 0 0 ...
## $ gravel         : int  0 0 0 0 0 0 0 0 0 0 ...
## $ garbage        : int  0 0 0 0 0 0 0 0 0 0 ...
## $ vaco_perc      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ vaco_num       : num  0 0 0 0 0 0 0 0 0 0 ...
## $ anco_perc      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ anco_num       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ arth_perc      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ arth_num       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ brra_perc      : int  0 0 0 0 0 0 0 0 0 0 ...
## $ brra_num       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ cabu_perc      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ cabu_num       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ cahi_perc      : num  0 0 0 0 0 0 0 0 0 0 ...
## $ cahi_num       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ cape_perc      : num  0 0 0 0 0 0 0 0 0 0 ...
```

```

## $ cape_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ cegl_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ cegl_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ clpa_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ clpa_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ crdo_perc     : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ crdo_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ drve_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ drve_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ epci_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ epci_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ gedi_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ gedi_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ hypo_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ hypo_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ lapu_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ lapu_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ lope_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ lope_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ madi_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ madi_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ moli_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ moli_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ mydi_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ mydi_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ myst_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ myst_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ pela_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ pela_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ pema_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ pema_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ plla_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ plla_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ poav_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ poav_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ prkn_perc     : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ prkn_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ raac_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ raac_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rara_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rara_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rasa_perc     : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rasa_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rare_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rare_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rasc_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rasc_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ ropa_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ ropa_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ ruac_perc     : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ ruac_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rula_perc     : num 0 0 0 0 0 0 0 0 0 0 0 ...
## $ rula_num      : int 0 0 0 0 0 0 0 0 0 0 0 ...
## $ ruur_perc     : int 0 0 0 0 0 0 0 0 0 0 0 ...

```

```
## $ ruur_num      : int  0 0 0 0 0 0 0 0 0 0 ...
## $ ruul_perc     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ ruul_num      : int  0 0 0 0 0 0 0 0 0 0 ...
## $ sara_perc     : int  0 0 0 0 0 0 0 0 0 0 ...
## $ sara_num      : int  0 0 0 0 0 0 0 0 0 0 ...
## $ scau_perc     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ scau_num      : int  0 0 0 0 0 0 0 0 0 0 ...
## $ stme_perc     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ stme_num      : int  0 0 0 0 0 0 0 0 0 0 ...
## [list output truncated]
```

```
veg$date <- ymd(veg$date)
```

```
#site_info
str(site_info)
```

```
## 'data.frame':   12 obs. of  6 variables:
## $ site_id      : chr  "bates_57b" "bks_56" "chong_68" "bks_68" ...
## $ area         : chr  "delta_1" "delta_1" "delta_2" "delta_2" ...
## $ grower       : chr  "George Bates" "Tejinder" "Nancy Chong" "Tejinder" ...
## $ company      : chr  "Bate's Farm" "BKS Farms" "Chong Farms" "BKS Farms" ...
## $ address      : chr  "Deltaport Way and 53 St" "2601 56th St" "6638 68th Street, Delta" "6570 68th st"
## $ cover_type   : chr  "grass" "bare" "grass" "bare" ...
```

```
#surveys
str(surveys)
```

```
## 'data.frame':   210 obs. of  3 variables:
## $ site : chr  "bates_57b" "bates_57b" "bates_57b" "bates_57b" ...
## $ round: chr  "2024_r1" "2024_r1" "2024_r1" "2024_r2" ...
## $ plots: chr  "p1" "p2" "d" "p1" ...
```

```
surveys <- surveys %>%
  rename(site_id = site)
surveys <- surveys %>%
  rename(plot_id = plots)
```

Add date of bee survey to the associated row in the survey data, based on site, plot and round.

```
# Extract relevant bee dates first
bee_dates <- bee %>%
  select(site_id, plot_id, round, date) %>%
  rename(bee_date = date)

#Make bee_dates file only contain one row for each site/plot/round/date combination
bee_dates_unique <- bee_dates %>%
  group_by(site_id, round) %>%
  summarise(bee_date = first(bee_date), .groups = "drop") #choose first date since all dates within a s
```

```

#Join bee_date onto survey data

surveys <- surveys %>% left_join(bee_dates_unique, by = c("site_id", "round"))

#Need to manually add in the dates for 2 surveys where I didn't find any bees (surjit_168 during 2025_r
missing_dates <- tibble::tribble(
  ~site_id, ~round, ~bee_date,
  "surjit_168", "2025_r2", as.Date("2025-05-13"),
  "bks_neaves", "2025_r1", as.Date("2025-04-14"),
  "bks_56", "2024_r3", as.Date("2025-06-21"),
  "bks_56", "2025_r3", as.Date("2025-06-18"),
  "bks_neaves", "2025_r3", as.Date("2025-06-24"),
  "surjit_168", "2025_r3", as.Date("2025-06-25"),
  "sandhu_ladnertrunk", "2025_r3", as.Date("2025-06-23") )

surveys <- surveys %>% rows_patch(missing_dates, by = c("site_id", "round"))

#add column for julian date
surveys$julian.date <- yday(surveys$bee_date)

```

Add area and cover type to survey dataframe based on site_id

```

surveys <- surveys %>%
  left_join(site_info %>% select(site_id, area, cover_type), by = c("site_id"))

```

Adding all the desired variables summarized at plot level as columns to the surveys dataframe

The desired variables are bee abundance for each caste of bees (queen and worker)

```

# 1. Summarize your actual bee observations
bee_counts <- bee %>%
  filter(caste %in% c("q", "w")) %>%
  group_by(site_id, round, plot_id, caste) %>%
  summarise(n_bees = n(), .groups = "drop")

# 2. Use your existing 'surveys' dataframe as the base
# This ensures you ONLY have rows for actual survey events
surveys <- surveys %>%
  # Create a slot for both 'q' and 'w' for every real survey row
  crossing(caste = c("q", "w")) %>%
  # Join the counts
  left_join(bee_counts, by = c("site_id", "round", "plot_id", "caste")) %>%
  # Fill the zeros for the surveys where that caste wasn't seen
  mutate(n_bees = replace_na(n_bees, 0))

```

Run model to see the effect of date, caste, and plot type on bee abundance.

1. Load packages and clean dataframe

```
#Load packages
library(DHARMA) #diagnostic tools for mixed regression models

## This is DHARMA 0.4.7. For overview type '?DHARMA'. For recent changes, type news(package = 'DHARMA')

library(glmmTMB) #fits generalized linear mixed models, including zero-inflated and overdispersed model.

#clean surveys dataframe and scale continuous predictors

#change character columns to factors
surveys <- surveys %>%
  mutate(
    site_id = as.factor(site_id),
    round = as.factor(round),
    plot_id = as.factor(plot_id),
    bee_date = as.Date(bee_date),
    area = as.factor(area),
    cover_type = as.factor(cover_type))

#add plot type column and make it a factor
surveys <- surveys %>%
  mutate(plot_type = if_else(plot_id == "d", "ditch", "field"))
surveys <- surveys %>%
  mutate(
    plot_type = as.factor(plot_type))

#scale continuous predictors
surveys$julian.date_scaled <- as.numeric(scale(surveys$julian.date))
surveys$n_bees_scaled <- as.numeric(scale(surveys$n_bees))
```

2. Try different random effects structures. Use zero-inflated model to account for high abundance of zeros in the dataset.

```
caste_glm_full <- glmmTMB(n_bees ~ caste*julian.date_scaled*plot_type +
  cover_type +
  (1|area/site_id/plot_id),
  ziformula = ~ caste*julian.date_scaled*plot_type +
  cover_type +
  (1|area/site_id/plot_id), # zero-inflation model
  family = poisson,
  data = surveys)

caste_glm_noArea <- glmmTMB(n_bees ~ caste*julian.date_scaled*plot_type +
  cover_type +
  (1|site_id/plot_id),
  ziformula = ~ caste*julian.date_scaled*plot_type +
```

```

        cover_type +
        (1|site_id/plot_id),
        family = poisson,
        data = surveys)

caste_glm_justSite <- glmmTMB(n_beets ~ caste*julian.date_scaled*plot_type +
        cover_type +
        (1|site_id),
        ziformula = ~ caste*julian.date_scaled*plot_type +
        cover_type +
        (1|site_id),
        family = poisson,
        data = surveys)

caste_glm_noRE <- glmmTMB(n_beets ~ caste*julian.date_scaled*plot_type +
        cover_type,
        ziformula = ~ caste*julian.date_scaled*plot_type +
        cover_type,
        family = poisson,
        data = surveys)

#compare models
anova(caste_glm_noRE, caste_glm_justSite, caste_glm_noArea, caste_glm_full)

```

```

## Data: surveys
## Models:
## caste_glm_noRE: n_beets ~ caste * julian.date_scaled * plot_type + cover_type, zi=~caste * julian.date_scaled
## caste_glm_justSite: n_beets ~ caste * julian.date_scaled * plot_type + cover_type + , zi=~caste * julian.date_scaled
## caste_glm_justSite:      (1 | site_id), zi=~caste * julian.date_scaled * plot_type + cover_type + (1 | site_id)
## caste_glm_noArea: n_beets ~ caste * julian.date_scaled * plot_type + cover_type + , zi=~caste * julian.date_scaled
## caste_glm_noArea:      (1 | site_id/plot_id), zi=~caste * julian.date_scaled * plot_type + cover_type + (1 | site_id/plot_id)
## caste_glm_full: n_beets ~ caste * julian.date_scaled * plot_type + cover_type + , zi=~caste * julian.date_scaled
## caste_glm_full:      (1 | area/site_id/plot_id), zi=~caste * julian.date_scaled * plot_type + cover_type + (1 | area/site_id/plot_id)
##
##          Df    AIC    BIC logLik deviance Chisq Chi Df Pr(>Chisq)
## caste_glm_noRE      18 919.42  992.14 -441.71   883.42
## caste_glm_justSite  20 920.16 1000.97 -440.08   880.16  3.253    2  0.1966
## caste_glm_noArea    22 892.53  981.41 -424.26   848.53 31.638    2 1.348e-07
## caste_glm_full      24 896.53  993.49 -424.26   848.53  0.000    2  1.0000
##
## caste_glm_noRE
## caste_glm_justSite
## caste_glm_noArea    ***
## caste_glm_full
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

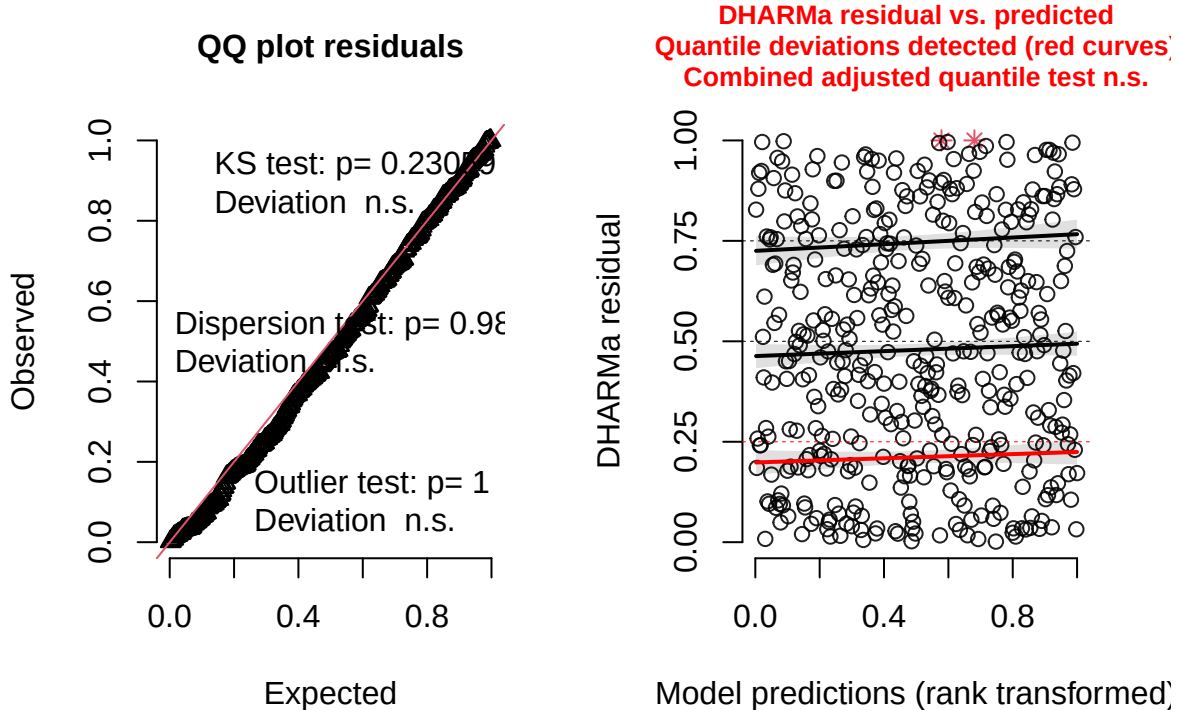
```
##no area model is best
```

2) Check diagnostics of model

- look good


```
#simulate residuals
residuals_caste_glm_noArea <- simulateResiduals(fittedModel = caste_glm_noArea, plot = TRUE)
```

DHARMA residual



3) Plot results

```
summary(caste_glm_noArea)
```

```
## Family: poisson ( log )
## Formula:
## n_beets ~ caste * julian.date_scaled * plot_type + cover_type +
## (1 | site_id/plot_id)
## Zero inflation:
## ~caste * julian.date_scaled * plot_type + cover_type + (1 | site_id/plot_id)
## Data: surveys
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    892.5    981.4   -424.3    848.5     398
##
## Random effects:
##
## Conditional model:
## Groups      Name      Variance Std.Dev.
## plot_id:site_id (Intercept) 3.099e-01 0.5567039
## site_id      (Intercept) 1.470e-07 0.0003834
```

```

## Number of obs: 420, groups:  plot_id:site_id, 36; site_id, 12
##
## Zero-inflation model:
##   Groups          Name          Variance Std.Dev.
##   plot_id:site_id (Intercept) 0.176769 0.42044
##   site_id          (Intercept) 0.003547 0.05956
## Number of obs: 420, groups:  plot_id:site_id, 36; site_id, 12
##
## Conditional model:
##                                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)                        0.3772     0.3162   1.193   0.23289
## castew                            1.2622     0.3895   3.240   0.00119
## julian.date_scaled                -0.5404     0.2472  -2.186   0.02880
## plot_typefield                     0.2256     0.3239   0.696   0.48615
## cover_typegrass                    0.0867     0.2542   0.341   0.73310
## castew:julian.date_scaled          -0.4306     0.4987  -0.864   0.38782
## castew:plot_typefield              -2.7103     0.4515  -6.003  1.94e-09
## julian.date_scaled:plot_typefield   0.4283     0.2817   1.520   0.12842
## castew:julian.date_scaled:plot_typefield  2.3910     0.5876   4.069  4.72e-05
##
## (Intercept)
## castew                            **
## julian.date_scaled                 *
## plot_typefield
## cover_typegrass
## castew:julian.date_scaled
## castew:plot_typefield              ***
## julian.date_scaled:plot_typefield
## castew:julian.date_scaled:plot_typefield ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)                       -0.2389     0.4861  -0.491  0.623104
## castew                            2.5760     0.6838   3.767  0.000165
## julian.date_scaled                 0.2944     0.5130   0.574  0.566088
## plot_typefield                     0.5404     0.5370   1.006  0.314235
## cover_typegrass                   -0.6817     0.4250  -1.604  0.108764
## castew:julian.date_scaled          -1.9886     0.7683  -2.588  0.009649
## castew:plot_typefield              -7.9207     2.5667  -3.086  0.002029
## julian.date_scaled:plot_typefield   1.1015     0.5940   1.854  0.063714
## castew:julian.date_scaled:plot_typefield  7.5828     2.5474   2.977  0.002914
##
## (Intercept)
## castew                            ***
## julian.date_scaled
## plot_typefield
## cover_typegrass
## castew:julian.date_scaled          **
## castew:plot_typefield              **
## julian.date_scaled:plot_typefield  .
## castew:julian.date_scaled:plot_typefield **
## ---

```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Use emmeans to look at estimates

```
library(emmeans)
```

```
## Welcome to emmeans.  
## Caution: You lose important information if you filter this package's results.  
## See '? untidy'
```

```
#compare the seasonal slopes for each caste in each plot type  
emtrends(caste_glm_noArea, ~ caste | plot_type, var = "julian.date_scaled")
```

```
## plot_type = ditch:  
## caste julian.date_scaled.trend SE df asymp.LCL asymp.UCL  
## q -0.540 0.247 Inf -1.025 -0.0559  
## w -0.971 0.437 Inf -1.828 -0.1139  
##  
## plot_type = field:  
## caste julian.date_scaled.trend SE df asymp.LCL asymp.UCL  
## q -0.112 0.137 Inf -0.381 0.1567  
## w 1.848 0.269 Inf 1.320 2.3761  
##  
## Results are averaged over the levels of: cover_type  
## Confidence level used: 0.95
```

```
#predicted counts for each group at the mean date  
emmeans(caste_glm_noArea, ~ caste * plot_type)
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

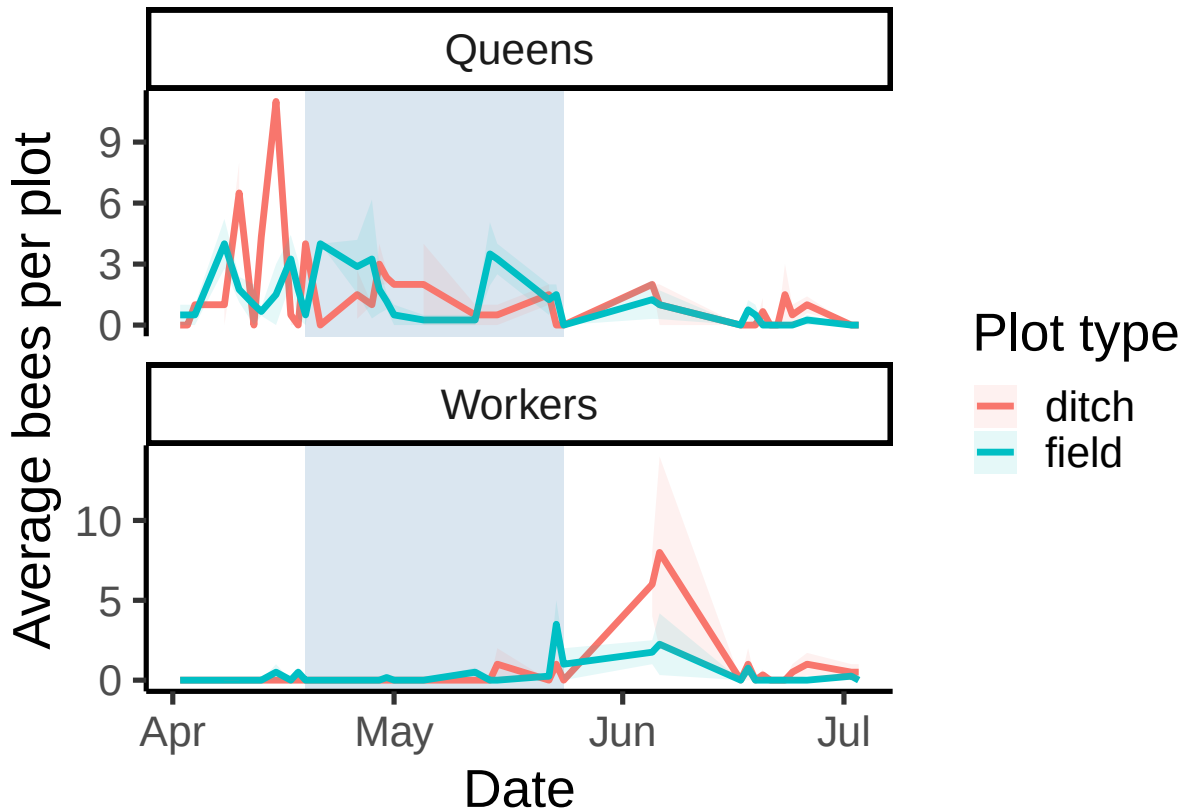
```
## caste plot_type emmean SE df asymp.LCL asymp.UCL  
## q ditch 0.421 0.275 Inf -0.118 0.959  
## w ditch 1.683 0.376 Inf 0.945 2.420  
## q field 0.646 0.189 Inf 0.276 1.017  
## w field -0.802 0.226 Inf -1.244 -0.360  
##  
## Results are averaged over the levels of: cover_type  
## Results are given on the log (not the response) scale.  
## Confidence level used: 0.95
```

Plots

```
#set peak bloom start and end dates  
bloom_start <- as.Date("2024-04-19")  
bloom_end <- as.Date("2024-05-24")
```

```
## Create date column
surveys <- surveys %>%
  mutate(
    # Convert Day of Year to an actual Date object (using 2024 as a reference year)
    date_formatted = as.Date(round(julian.date) - 1, origin = "2024-01-01")
  )

ggplot(surveys, aes(x = date_formatted, y = n_bees, color = plot_type)) +
  # Add the bloom window as a shaded background
  annotate("rect", xmin = bloom_start, xmax = bloom_end, ymin = -Inf, ymax = Inf,
    alpha = .2, fill = "steelblue") +
  stat_summary(fun = mean, geom = "line", linewidth = 1.2) +
  stat_summary(fun.data = mean_se, geom = "ribbon", alpha = 0.1, aes(fill = plot_type), color = NA) +
  facet_wrap(~ caste, ncol = 1, scales = "free_y", labeller = as_labeller(c("w" = "Workers", "q" = "Queens"))) +
  labs(x = "Date", y = "Average bees per plot", color = "Plot type", fill = "Plot type") +
  theme_classic(base_size = 20)
```



Plot behaviours of each caste in each plot type

```
#add plot type column to bee dataframe
bee <- bee %>%
  mutate(plot_type = if_else(plot_id == "d", "ditch", "field"))
```

```

bee <- bee %>%
  mutate(
    plot_type = as.factor(plot_type))
# Summary data for behavior proportions
behavior_summary <- bee %>%
  filter(caste %in% c("q", "w")) %>%
  group_by(plot_type, caste, behaviour) %>%
  summarise(total_count = n(), .groups = "drop") %>%
  group_by(plot_type, caste) %>%
  mutate(proportion = total_count / sum(total_count))

# Plotting the breakdown
ggplot(behavior_summary, aes(x = plot_type, y = proportion, fill = behaviour)) +
  geom_bar(stat = "identity", position = "fill") +
  facet_wrap(~caste, labeller = as_labeller(c("w" = "Workers", "q" = "Queens")) +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_discrete(labels = c(
    "flying" = "Flying",
    "foraging" = "Foraging",
    "nest_searching" = "Nest searching",
    "resting" = "Resting"
  ))) +
  labs(
    x = "Plot type",
    y = "Percentage of observations",
    fill = "Behaviour"
  ) +
  theme_classic(base_size = 15)

```

